

Minimum Viable Product (MVP)

Motor Inteligente de Segmentación de Clientes

Alan Alejandro Betancourt Ramírez

10 de septiembre de 2026

Contents

1. Objetivo del MVP	3
2. Decisión arquitectónica principal	3
3. Alcance del MVP	3
4. Fuente CRM simulada	4
5. Datos simulados	4
6. Contrato interno del MVP	4
6.1 Customer	4
6.2 Transaction	5
6.3 Interaction	5
6.4 CampaignEvent	5
7. Adaptador de la fuente simulada	5
8. Persistencia en PostgreSQL	5
9. Preparación de datos	6
10. Features de segmentación	6
11. Modelo de Machine Learning	6
12. Validación e interpretación del modelo	7
13. Azure Machine Learning	7
14. Backend principal con FastAPI	7
15. Recomendaciones con Azure OpenAI	8
16. Frontend con React	8
Dashboard	8
Clientes	8

Segmentos	9
Detalle de segmento	9
17. Requerimientos funcionales del MVP	9
18. Requerimientos no funcionales del MVP	9
19. Tecnologías del MVP	10
20. Funcionalidades fuera del MVP	10
21. Criterios de aceptación	11
22. Plan de desarrollo de 3 semanas	11
Semana 1 - Datos y núcleo analítico	11
Semana 2 - Backend y servicios de IA	11
Semana 3 - Frontend, pruebas y entrega	12
23. Prioridades	12
24. Supuestos del MVP	12
Empresa: PluriOne S.A. de C.V.	
Nombre comercial: Develop Talent & Technology	
Proyecto: Motor Inteligente de Segmentación de Clientes	
Duración objetivo: 3 semanas	
Versión: 1.1	

1. Objetivo del MVP

Desarrollar una primera versión funcional del Motor Inteligente de Segmentación de Clientes que demuestre el ciclo principal del producto utilizando datos simulados.

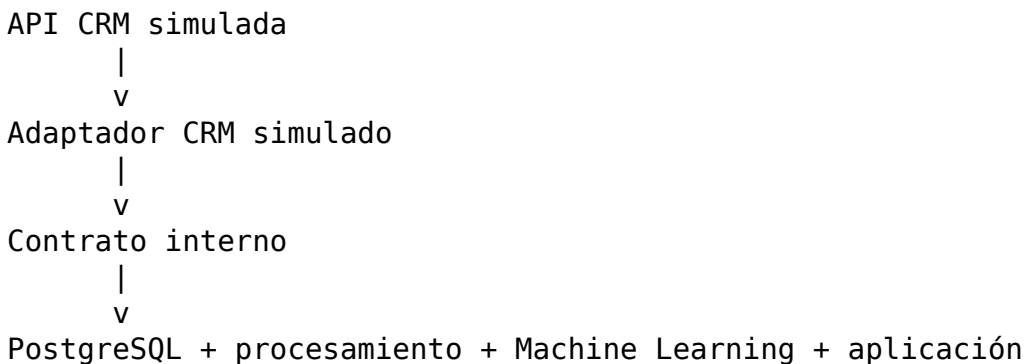
El MVP deberá demostrar que una fuente CRM simulada puede entregar datos, que esos datos pueden adaptarse a un contrato interno estable, procesarse, utilizarse para generar segmentos mediante Machine Learning y visualizarse desde una aplicación web con recomendaciones comerciales por segmento.

2. Decisión arquitectónica principal

El sistema no se construirá alrededor del CRM simulado.

La fuente simulada será tratada como un sistema externo. El núcleo del proyecto trabajará únicamente con un contrato interno de datos.

Se implementará un adaptador específico para la fuente simulada. En el futuro, un CRM real podrá integrarse creando otro adaptador, sin modificar el motor de segmentación mientras el contrato interno permanezca compatible.



3. Alcance del MVP

El MVP deberá incluir:

- generación de datos simulados realistas;
- una API independiente que simule la fuente de datos de un CRM;
- un adaptador que transforme la respuesta de esa API al contrato interno;
- validación y persistencia de datos normalizados;
- cálculo de features comerciales;
- modelo de clustering desarrollado en Python;
- interpretación de clusters;
- despliegue del modelo en Azure Machine Learning;
- backend principal con FastAPI;
- recomendaciones por segmento mediante Azure OpenAI;
- frontend en React;
- pruebas básicas y documentación.

4. Fuente CRM simulada

Debido a que no existe acceso a un CRM real, se implementará una API que represente una fuente externa de datos.

La API simulada deberá comportarse como un sistema independiente del backend principal. Sus datos podrán utilizar nombres y estructuras distintos al contrato interno para demostrar el funcionamiento real del adaptador.

Endpoints mínimos propuestos:

- GET /crm/customers
- GET /crm/transactions
- GET /crm/interactions
- GET /crm/campaign-events

La API simulada podrá alimentarse desde archivos semilla o datos generados mediante Python.

5. Datos simulados

El dataset deberá ser suficientemente realista para producir patrones diferenciables.

Deberá incluir, cuando sea posible:

- clientes con diferente antigüedad;
- diferentes frecuencias de compra;
- diferentes niveles de gasto;
- categorías de compra;
- comportamiento de campañas;
- interacciones digitales;
- clientes nuevos;
- clientes frecuentes;
- clientes de alto valor;
- clientes con baja actividad.

No se generarán únicamente valores aleatorios sin estructura. Los datos deberán permitir validar si el modelo logra descubrir comportamientos coherentes.

6. Contrato interno del MVP

El adaptador deberá convertir la fuente simulada a un modelo interno mínimo.

6.1 Customer

- customer_id
- age
- city
- registered_at

6.2 Transaction

- transaction_id
- customer_id
- purchased_at
- amount
- category

6.3 Interaction

- interaction_id
- customer_id
- interaction_type
- occurred_at

6.4 CampaignEvent

- campaign_id
- customer_id
- event_type
- occurred_at

Los tipos y obligatoriedad de los campos deberán documentarse en el código.

7. Adaptador de la fuente simulada

Se implementará un SimulatedCRMAdapter o componente equivalente.

Será responsable de:

- consultar la API simulada;
- mapear nombres externos a nombres internos;
- convertir fechas y tipos de datos;
- normalizar valores;
- validar campos requeridos;
- reportar registros inválidos;
- entregar estructuras compatibles con el contrato interno.

El adaptador no deberá calcular clusters ni contener lógica del modelo de ML.

8. Persistencia en PostgreSQL

PostgreSQL almacenará los datos normalizados y resultados del sistema.

Entidades mínimas:

- customers
- transactions
- interactions
- campaign_events

- segmentation_runs
- segments
- customer_segments
- recommendations

Los datos externos deberán pasar por el adaptador antes de persistirse como información interna.

9. Preparación de datos

Python será utilizado para limpiar y preparar la información.

El MVP deberá manejar como mínimo:

- duplicados;
- valores faltantes;
- tipos de datos;
- fechas;
- categorías inconsistentes;
- registros inválidos;
- escalado o normalización cuando el algoritmo lo necesite.

10. Features de segmentación

Las features mínimas serán RFM:

- **Recency:** días desde la última compra.
- **Frequency:** número de compras.
- **Monetary:** gasto acumulado.

También podrán incorporarse:

- ticket promedio;
- aperturas de correo;
- clics en campañas;
- visitas;
- carritos abandonados;
- categoría favorita;
- métricas simples de engagement.

Las features adicionales solo se usarán si aportan valor claro al modelo.

11. Modelo de Machine Learning

El modelo se desarrollará en Python con Scikit-learn.

Enfoque inicial:

- clustering no supervisado;
- algoritmo inicial: K-Means.

El modelo deberá:

- recibir features preparadas;
- generar clusters;
- asignar cada cliente a un cluster;
- permitir analizar los centroides o estadísticas de cada grupo.

El modelo se probará localmente antes de utilizar Azure Machine Learning.

12. Validación e interpretación del modelo

Se deberán probar distintas cantidades de clusters y revisar:

- Silhouette Score;
- Inertia;
- distribución de clientes;
- estabilidad de resultados;
- interpretación comercial.

Los clusters se mantendrán inicialmente como Cluster 0, Cluster 1, etc.

Solo después del análisis podrán recibir etiquetas como:

- clientes frecuentes de alto valor;
- clientes nuevos;
- clientes ocasionales;
- clientes con riesgo de inactividad.

13. Azure Machine Learning

Una vez validado localmente, el modelo deberá registrarse y desplegarse en Azure Machine Learning.

Se utilizará para:

- registrar el modelo;
- mantener una versión;
- definir dependencias;
- crear un deployment;
- exponer un endpoint de inferencia.

El endpoint de Azure ML será consumido desde el backend principal, no desde React.

14. Backend principal con FastAPI

El backend principal será responsable de la lógica de aplicación.

Endpoints iniciales propuestos:

- POST /ingestion/sync
- GET /customers
- GET /customers/{id}

- POST /segmentation/run
- GET /segments
- GET /segments/{id}
- POST /segments/{id}/recommendation

Responsabilidades:

- ejecutar el adaptador;
- guardar datos normalizados;
- consultar PostgreSQL;
- preparar solicitudes al modelo;
- consumir Azure ML;
- consumir Azure OpenAI;
- devolver respuestas al frontend;
- proteger credenciales.

15. Recomendaciones con Azure OpenAI

Azure OpenAI recibirá un resumen agregado de cada segmento, por ejemplo:

- recency promedio;
- frequency promedio;
- monetary promedio;
- ticket promedio;
- categoría dominante;
- engagement promedio.

Deberá generar una descripción y una recomendación comercial por segmento.

Las recomendaciones estarán dirigidas al segmento completo, no a clientes individuales.

Azure OpenAI no determinará los clusters.

16. Frontend con React

El frontend será desarrollado con React, preferentemente TypeScript/TSX.

Vistas mínimas:

Dashboard

- total de clientes;
- total de segmentos;
- distribución de clientes;
- métricas básicas.

Clientes

- listado básico;
- segmento asignado;
- datos comerciales relevantes.

Segmentos

- nombre o etiqueta interpretada;
- cantidad de clientes;
- métricas promedio.

Detalle de segmento

- clientes del grupo;
- características principales;
- recomendación comercial.

React consumirá únicamente la API del backend principal.

17. Requerimientos funcionales del MVP

- RF-MVP-01.** El sistema deberá contar con una API que simule una fuente CRM externa.
- RF-MVP-02.** La fuente simulada deberá exponer clientes, transacciones, interacciones y eventos de campaña.
- RF-MVP-03.** El sistema deberá contar con un adaptador para la fuente CRM simulada.
- RF-MVP-04.** El adaptador deberá transformar la respuesta externa al contrato interno.
- RF-MVP-05.** El sistema deberá validar los datos adaptados.
- RF-MVP-06.** El sistema deberá persistir datos normalizados en PostgreSQL.
- RF-MVP-07.** El sistema deberá generar features RFM.
- RF-MVP-08.** El sistema deberá permitir agregar features complementarias justificadas.
- RF-MVP-09.** El sistema deberá ejecutar un modelo de clustering.
- RF-MVP-10.** El sistema deberá asignar un cluster a cada cliente procesado.
- RF-MVP-11.** El sistema deberá almacenar ejecuciones y resultados.
- RF-MVP-12.** El sistema deberá permitir consultar segmentos y clientes asociados.
- RF-MVP-13.** El sistema deberá mostrar estadísticas principales de cada segmento.
- RF-MVP-14.** El modelo deberá poder desplegarse mediante Azure Machine Learning.
- RF-MVP-15.** El backend deberá consumir el endpoint de Azure ML.
- RF-MVP-16.** El sistema deberá generar una recomendación comercial por segmento mediante Azure OpenAI.
- RF-MVP-17.** El frontend deberá consumir únicamente la API del backend principal.
- RF-MVP-18.** El sistema deberá permitir volver a sincronizar datos y repetir la segmentación.
- RF-MVP-19.** El motor de segmentación no deberá depender directamente del esquema de la API simulada.

18. Requerimientos no funcionales del MVP

- RNF-MVP-01. Desacoplamiento:** la fuente externa y el motor deberán mantenerse separados mediante el adaptador.
- RNF-MVP-02. Adaptabilidad:** una nueva fuente deberá poder integrarse creando un nuevo adaptador.
- RNF-MVP-03. Seguridad:** credenciales y claves de servicios externos deberán permanecer en el backend.

RNF-MVP-04. Privacidad: se utilizarán únicamente datos simulados.

RNF-MVP-05. Modularidad: simulador, adaptador, persistencia, procesamiento, ML, backend y frontend deberán mantenerse separados por responsabilidad.

RNF-MVP-06. Manejo de errores: datos inválidos o fallos externos deberán tratarse de forma controlada.

RNF-MVP-07. Mantenibilidad: el código deberá organizarse por módulos claros.

RNF-MVP-08. Trazabilidad: las ejecuciones de ingestión y segmentación deberán poder identificarse.

RNF-MVP-09. Contrato estable: cambios incompatibles en el modelo interno deberán documentarse y versionarse.

19. Tecnologías del MVP

Principales:

- Python;
- FastAPI;
- PostgreSQL;
- React;
- TypeScript/TSX;
- Pandas;
- NumPy;
- Scikit-learn;
- Azure Machine Learning;
- Azure OpenAI Service;
- Git/GitHub.

Opcional si el tiempo lo permite:

- Docker.

20. Funcionalidades fuera del MVP

No serán obligatorias:

- conexión a CRM real;
- adaptadores para múltiples CRM reales;
- integración bidireccional con CRM;
- ERP real;
- Google Analytics real;
- procesamiento en tiempo real;
- campañas automáticas;
- Power BI completo;
- Azure AI Search;
- Microsoft Entra ID;
- GitHub Actions;
- monitoreo avanzado;
- reentrenamiento automático;
- churn prediction;

- CLV avanzado;
- infraestructura de producción completa.

21. Criterios de aceptación

El MVP se considerará completo cuando pueda demostrar de extremo a extremo que:

1. la API CRM simulada entrega datos;
2. el adaptador consume y transforma esos datos;
3. la salida cumple el contrato interno;
4. los datos normalizados se almacenan en PostgreSQL;
5. se calculan features RFM correctamente;
6. el modelo local genera clusters diferenciados;
7. los clusters se evalúan e interpretan;
8. el modelo se registra y despliega en Azure ML;
9. el backend puede consumir el endpoint de Azure ML;
10. los resultados se almacenan y consultan mediante FastAPI;
11. Azure OpenAI genera una recomendación por segmento;
12. React muestra clientes, segmentos y recomendaciones;
13. las credenciales permanecen fuera del frontend;
14. el motor de segmentación funciona sin conocer el formato nativo de la API CRM simulada.

22. Plan de desarrollo de 3 semanas

Semana 1 - Datos y núcleo analítico

- definir contrato interno v1;
- diseñar dataset simulado;
- crear API CRM simulada;
- implementar SimulatedCRMAdapter;
- crear esquema inicial de PostgreSQL;
- validar y persistir datos;
- calcular RFM;
- crear y validar primer modelo K-Means;
- interpretar resultados iniciales.

Semana 2 - Backend y servicios de IA

- crear backend principal con FastAPI;
- implementar endpoints de ingestión, clientes y segmentos;
- almacenar ejecuciones;
- registrar modelo en Azure ML;
- crear deployment y endpoint;
- conectar FastAPI con Azure ML;
- integrar Azure OpenAI para recomendaciones;
- realizar pruebas de integración.

Semana 3 - Frontend, pruebas y entrega

- crear React/TypeScript;
- implementar dashboard;
- crear vistas de clientes y segmentos;
- crear detalle de segmento y recomendaciones;
- integrar frontend con backend;
- mejorar validaciones y manejo de errores;
- realizar pruebas completas;
- documentar arquitectura y uso;
- preparar demostración final.

23. Prioridades

Si el tiempo es limitado, el orden de prioridad será:

1. contrato interno;
2. API CRM simulada;
3. adaptador;
4. persistencia y preparación de datos;
5. RFM y modelo de clustering;
6. interpretación de segmentos;
7. backend principal;
8. Azure Machine Learning;
9. React;
10. Azure OpenAI;
11. mejoras visuales y extras.

El núcleo de integración y segmentación tendrá prioridad sobre funcionalidades secundarias.

24. Supuestos del MVP

- No habrá acceso directo a los sistemas reales de la empresa.
- No se utilizarán datos reales de clientes.
- La fuente CRM será simulada mediante una API propia.
- La API CRM simulada será independiente del backend principal.
- El sistema contará con un adaptador para esa fuente.
- El contrato interno será la única estructura que deberá conocer el motor de segmentación.
- El primer modelo se basará como mínimo en RFM y K-Means.
- Azure Machine Learning se utilizará después de validar el modelo localmente.
- Azure OpenAI se utilizará únicamente para interpretación y recomendaciones.
- La futura integración con un CRM real se resolverá mediante un nuevo adaptador.
- El objetivo del MVP es demostrar viabilidad técnica, arquitectura adaptable y valor funcional en aproximadamente tres semanas.